How SCRATCH calls copy-number variation

inferCNV and Numbat, exactly as the pipeline runs them

2026-10-29

CONTENTS

1 Why this document exists
2 The idea both callers share
3 inferCNV in SCRATCH
  3.1 Where it sits
  3.2 Step 1 — resolve the annotation column
  3.3 Step 2 — decide which labels are normal
  3.4 Step 3 — optional capping
  3.5 Step 4 — the gene order file
  3.6 Step 5 — the actual calls
  3.7 Step 6 — turning the output into one number per cell
  3.8 What inferCNV structurally cannot see
4 Numbat in SCRATCH
  4.1 Three processes, and why
  4.2 Step 1 — group by individual, not by library
  4.3 Step 2 — barcodes and namespacing
  4.4 Step 3 — build the expression reference
  4.5 Step 4 — pileup and phasing
  4.6 Step 5 — the model
  4.7 Step 6 — the per-cell call
  4.8 Step 7 — what gets published
  4.9 Reading the two percentages
5 Where the two callers meet
  5.1 Each caller is normalised to a per-cell verdict
  5.2 The consensus
  5.3 The lineage gate
6 Side by side
7 Parameters that change the answer
8 Traps worth knowing
9 Worked example: GBM1-DFCI3

1 Why this document exists

Two of SCRATCH's CNV callers answer the same question — *is this cell malignant?* — from completely different evidence, and they are configured differently enough that the same cell can get different answers for reasons that have nothing to do with biology. This is the reference for what each one actually does in this pipeline: not what the published method can do, but which arguments SCRATCH passes, which cells it treats as normal, and which number ends up in the final object.

Everything here is stated from the code. Where a line of code settles a question, the file and the line are named so you can check it.

> **The one-sentence version.** inferCNV looks for expression shifted along the genome and needs you to *name the normal cells*; Numbat looks for allele imbalance at phased SNPs and needs you to *hand it a normal expression profile*. Neither of them can tell you a sample has no tumour in it, which is why SCRATCH applies a lineage gate on top of both.

2 The idea both callers share

A copy-number gain means more DNA, which usually means more RNA from every gene in that region; a loss means less. Neither caller can see absolute copy number in a single cell. Both work by **comparison**: they measure a cell against a baseline built from cells believed to be diploid, and look for runs of neighbouring genes that are consistently shifted in the same direction.

That makes the baseline the single most important input. If the baseline contains tumour cells, the tumour's own aberrations are subtracted away and the caller goes quiet. If the baseline is too small or comes from a different experiment, ordinary batch variation looks like copy number.

The two callers differ in *how you specify that baseline*, and this is the difference that causes the most confusion:

	inferCNV	**Numbat**
What you supply	**Cells** — a per-cell group label plus the names of the groups that are normal	**A profile matrix** — genes × pseudobulk columns of normalised expression
Argument	`ref_group_names`	`lambdas_ref`
Are the reference cells excluded from scoring?	**Yes.** They are the reference; the rest are "observations"	**No.** Nothing links `lambdas_ref` back to `count_mat`, so the same cells are still scored
Can it use a DNA normal (WGS/WES)?	No	Not as the baseline. Only as `segs_consensus_fix` or prephased SNPs

The second row is the one people expect to be the same and it is not. **In Numbat there is no `ref_group_names`, no `normal_cells` and no annotation file.** Verified against the installed package (38 formals on `run_numbat`) and the CRAN 1.5.2 manual (40 formals; the two additions are `gtf` and `plot_min_depth`).

3 inferCNV in SCRATCH

3.1 Where it sits

One process, `INFERCNV` (`modules/local/infercnv/main.nf`), labelled `process_high` and `process_high_memory`. It renders `modules/local/infercnv/notebook_infercnv.qmd` with `quarto render`, so the stage's report *is* its log. It runs **once over the whole merged object** — there is no per-patient split — and it is on by default (`--skip_infercnv` turns it off).

3.2 Step 1 — resolve the annotation column

The notebook does not trust a single column name. It tries, in order:

```
--celltype_col, azimuth_labels, celltype, cell_type, annotation,
sctype, sctype_agg, sctype_labels, cell_label
```

and stops with the vocabulary it found if none exists. The chosen column is copied to `reference_label`, and cells with `NA` or an empty label become `"Unassigned"` — inferCNV needs a group for every cell, and `NA` is not a group.

3.3 Step 2 — decide which labels are normal

`--stratify_reference_labels` is split on `;`, `,` or `|`. The matching is **not** `%in%`: it goes through `scratch_match_labels()` from `assets/viz/scratch_report.R`, the same matcher STRATIFY and CNV_CONCORDANCE use, so `T_Cells` also matches `T cell`. If that helper was not staged the notebook refuses to run rather than silently matching differently from the rest of the pipeline.

Two guards then fire before inferCNV is touched at all:

- **no reference label matched** → stop, printing every label actually present;
- **every cell is a reference cell** → stop, because there is nothing left to observe.

Both of these used to surface as `obs_dendrogram[[1]]: subscript out of bounds` from deep inside inferCNV's plotting code.

> **The bug this replaced, because it is instructive.** An earlier version carried a `cell_types_to_keep` whitelist and subset the object to it. On the scType branch the list kept `Epithelial`, to which scType assigns *zero* cells — so all 12,871 surviving cells were reference cells, there were no observations, and the stage died. The tumour cells were in the 4,308 cells labelled `Unknown`, which the whitelist had thrown away. Whitelisting observations is the wrong shape: you cannot know in advance what the malignant cells were called, and that is the whole question.
> **SCRATCH now keeps every cell, names only the reference, and lets everything else be observed.**

3.4 Step 3 — optional capping

`--cnv_infercnv_max_cells_per_group` (default `0` = off) subsamples any group over the limit, with a fixed seed (`022003`) so a retry sees the same cells. Capping happens *before* the checks above, so every count the report prints describes the object inferCNV really saw. Cells not sampled get **no inferCNV score at all**.

3.5 Step 4 — the gene order file

Built from `annotables::grch38` , filtered to `biotype == "protein_coding"` and chromosomes 1–22, sorted by position, deduplicated on symbol, with `chr` prefixed. **Human only** — there is no mouse branch here. Sex chromosomes are excluded, so no CNV is ever reported on X or Y by this caller.

3.6 Step 5 — the actual calls

```
infercnv_object <- infercnv::CreateInfercnvObject(
  raw_counts_matrix = seurat_counts,       # the "counts" layer
  annotations_file  = reference_table,     # one column: reference_label
  delim             = "\t",
  gene_order_file   = infercnv_genes,
  ref_group_names   = .ref_found)          # labels PRESENT, not labels requested

infercnv_object <- infercnv::run(
  infercnv_object,
  cutoff            = 0.25,
  out_dir           = './data/infercnv',
  cluster_by_groups = TRUE,
  analysis_mode     = input_analysis_mode, # --cnv_analysis_mode, default 'samples'
  denoise           = TRUE,
  HMM               = TRUE,
  BayesMaxPNormal   = bayes_max_p_normal,  # --cnv_infercnv_bayes_max_p_normal, default 0
  num_threads       = n_threads)           # task.cpus, not the notebook default of 8
```

Two of these deserve explanation.

`ref_group_names = .ref_found` passes the labels that **exist**, not the ones requested. Naming a group with no cells makes inferCNV build an empty reference and fail obscurely much later.

`BayesMaxPNormal = 0` **disables** inferCNV's Bayesian mixture filter. That is a deliberate workaround, not a tuning choice. With it on, the run aborts inside inferCNV's own MCMC code:

```
Error in `do.call()`: second argument must be a list
  infercnv::run -> inferCNVBayesNet -> runMCMC -> getProbabilities
    -> do.call(rbind, mcmc[[j]])
```

`getProbabilities` assumes every cell group accumulated MCMC draws; a group that produced none leaves `NULL` , and `rbind` refuses it. It is a property of the grouping, so no retry can get around it. With the filter off, the HMM's own state calls stand — which is what `add_to_seurat` and the STRATIFY consensus read anyway. Restore the upstream default with `--cnv_infercnv_bayes_max_p_normal 0.5` once the image carries a version that tolerates an empty group.

3.7 Step 6 — turning the output into one number per cell

`infercnv::add_to_seurat(top_n = 10)` writes roughly 220 metadata columns: `has_cnv_chrN`, `proportion_cnv_chrN`, `proportion_scaled_cnv_chrN` and the loss/dupli variants, for each autosome. The notebook reduces them to two:

```
genomwide_cnv_proportion <- log1p(sum of all proportion_scaled_cnv_* columns)
n_affected_chromosomes   <- sum of all has_cnv_* columns
```

Note the spelling: **`genomwide_cnv_proportion`**, missing the "e". It is the column name in the object; grep for it as written.

This number is a **continuous CNV load**, not a call. inferCNV by itself never says "malignant" in SCRATCH — the threshold is applied later, in STRATIFY — see *Where the two callers meet*.

3.8 What inferCNV structurally cannot see

Copy-neutral loss of heterozygosity. One parental copy is lost and the other duplicated: the allele ratio changes, the dosage does not, so there is no expression signal to smooth. This is the gap Numbat exists to fill, and it is why SCEVAN was removed in 2026-08 — it was expression-only too, so a third expression caller added agreement without adding information.

4 Numbat in SCRATCH

4.1 Three processes, and why

Process	Image	Input → Output
`NUMBAT_PREP`	`scratch-cnv` (Seurat, **no** numbat)	annotated object → per-library barcodes, pooled counts, cell map, reference-cell list
`NUMBAT_PILEUP`	numbat image	BAM + barcodes → phased allele counts, per individual
`NUMBAT_RUN`	numbat image (numbat, **no** Seurat)	counts + alleles → per-cell calls

They are separate so the expensive pileup caches independently of the model fit: retuning `--numbat_t` or `--numbat_max_iter` re-runs only `NUMBAT_RUN`. The image split is also why PREP *selects* the reference cells and RUN *aggregates* them — `aggregate_counts()` needs the numbat package, which is not in the Seurat image.

Numbat is **on by default** (`run_numbat = true`). It is the better caller — it does not depend on a reference lineage being correctly named to build its null, which is the failure mode that once let a fibroblast-only null set inferCNV's cutoff to zero and call whole immune lineages malignant. It is not the cheaper one: it needs the aligned BAM (~80 GB per sample here) plus a ~10 GB phasing panel, and the pileup dominates the stage — measured 3 h against inferCNV's 44 min on one machine. A run with no BAM is not an error; `main.nf` switches Numbat off with a warning.

4.2 Step 1 — group by individual, not by library

`bin/numbat_prep.R` splits the object by **individual**, resolving the group column from `--numbat_group_col`, then `patient_id`, `patient`, `subject_id`, `donor_id`. With none of them present it falls back to the library column and says so loudly, because that is the configuration this script exists to prevent.

The reason is a property of the method, not a preference:

> **`numbat:::mark_tumor_lineage()` has no null.** It assigns `tumor_root = mut_nodes[which.max(mut_burdens)]` — the heaviest clade is always labelled `tumor`. A library with no malignant cells therefore **cannot** come back empty; it comes back with its own noise partitioned into "tumour" and "normal". Pooling an individual's libraries gives the real clones something to dominate.

This is the direct cause of the "the PBMCs look strange" reading: run alone, a blood library is *guaranteed* to report neoplastic cells.

4.3 Step 2 — barcodes and namespacing

The BAM speaks raw 16 bp cellranger barcodes; the merged object does not. `raw_barcode()` recovers the barcode from `PREFIX_AACGT-1`, `AACGT-1_1` or a bare `AACGT-1`, and the pooled cell id becomes `<library>_<barcode>`. That is what Numbat sees in both the count matrix and `df_allele`, and it is unique across the cohort — two libraries sharing a raw barcode is expected and harmless. A barcode duplicated *within* one library is a genuine ambiguity, and all copies are dropped.

4.4 Step 3 — build the expression reference

This is the part most often misunderstood, so it is worth stating plainly.

`lambdas_ref` is a matrix, not a cell list. The documentation: *"Either a named vector with gene names as names and normalized expression as values, or a matrix where rownames are genes and columns are pseudobulk names."* Numbat matches each pseudobulk against the **columns** and picks the best fit.

SCRATCH builds it the way the package author's own worked example does (`vignettes/mouse.Rmd` §4):

```
clusters  <- p2$clusters$PCA$multilevel
ref_annot <- data.frame(cell = names(clusters), group = unname(clusters))
ref       <- aggregate_counts(count_mat_ref,
                 ref_annot %>% group_by(group) %>% filter(n() > 100))
```

Two things to notice. The columns are **raw unsupervised clusters with no biological names at all**, and the floor is **100 cells**. What a column needs is transcriptional homogeneity and enough cells — not an identity.

SCRATCH therefore groups by `<label>__c<cluster>`, not by label:

- Eligibility is still the label rule (`--stratify_reference_labels`), so no malignant cell enters the diploid baseline.
- Granularity is the cluster (`--numbat_cluster_col`, default `seurat_clusters`).

- The floor is per **column** (`--numbat_ref_min_group_cells` , default `100`).

Why this matters: **scType assigns one label per cluster**, not per cell. It sums scores within a cluster, takes the top one, and stamps it on every cell. So grouping by label collapses several clusters into one averaged column and discards resolution the clustering already produced, for nothing. On real data this took a 3-column reference to 4 columns.

With no cluster column resolved, the reference degrades to one column per cell type — the old behaviour — with a warning.

4.4.1 The glioma conflict, and the opt-in

The maintainer gives two rules that cannot both be satisfied in glioma (kharchenkolab/numbat#224):

Rule	What SCRATCH does
1. Build with `aggregate_counts()` **from raw counts** — *"Don't use Seurat normalization"*	PREP reads `layer = "counts"` ; RUN passes `normalized = TRUE` , which means *return* normalised values
2. *"Ensure the reference includes the relevant normal cell types present in your sample"*	The 12 immune/stromal labels; see the caveat below
3. *"Make sure no tumor/malignant cells are included in your diploid reference"*	Eligibility is the reference-label list, so `Malignant` can never enter
4. Internal normals from the same experiment *"often works better than external datasets"*	An internal reference whenever ≥ `--numbat_min_ref_cells` exist; `numbat::ref_hca` only as fallback
5. Raise `min_LLR` / lower `max_entropy` against false positives in normals	Both exposed; both left at Numbat's own defaults

Rules 2 and 3 conflict: GBM tissue contains normal astrocytes, OPCs and neurons, but the Neftel malignant states carry **exactly those labels**. No label-based rule can separate them. So `Astrocyte` , `OPC` , `Radial_Glia` and `Neuron` are deliberately left out of the reference list, and the only route in is `--numbat_extra_reference_clusters` — an explicit, per-cluster, human judgement, **empty by default**, because deciding it from the annotation alone would be circular.

4.4.2 The reference cells are not held out

Nothing connects `lambdas_ref` back to `count_mat` . The cells that built the baseline are still in the matrix, still scored, and **can still be called tumour**. That is intentional: removing them removes the diploid anchor the phylogeny needs. It is also precisely why the lineage-gated percentage, not the raw one, is the number to read for a blood library.

4.5 Step 4 — pileup and phasing

`NUMBAT_PILEUP` runs the package's own `pileup_and_phase.R` : `cellsnp-lite` counts reads at known SNP positions in each BAM, then Eagle2 phases them against the 1000 Genomes panel, one chromosome at a time.

> **Pooling costs wall time.** `pileup_and_phase.R` writes **one `cellsnp-lite` command per sample** into `run_pileup.sh` and then runs the script. Each command uses all `task.cpus` internally, so cores are not idle — but the samples are **serial**. Grouping 17 libraries into one individual turns 17 parallel pileups into one pileup of roughly 17×. `conf/base.config` allows 120 h for PILEUP and 192 GB / 96 h for RUN; both are *estimates* from an observed failure pattern, not measurements.

4.6 Step 5 — the model

```
ref <- numbat::aggregate_counts(count_mat[, annot$cell], annot, normalized = TRUE)

numbat::run_numbat(
  count_mat   = count_mat,      # ALL cells, reference included
  lambdas_ref = ref,            # genes x cluster-columns
  df_allele   = df_allele,      # phased allele counts
  genome      = "hg38",
  t           = 1e-5,           # --numbat_t
  ncores      = task.cpus,
  max_iter    = 2,              # --numbat_max_iter
  min_LLR     = 5,              # --numbat_min_llr
  max_entropy = 0.5)            # --numbat_max_entropy
```

Measured defaults in the installed package: `lambdas_ref` is **required**, `common_diploid = TRUE`, `diploid_chroms = NULL`, `exclude_neu = TRUE`, `min_LLR = 5`. `numbat::ref_hca`, the external fallback, is 24,756 genes × 12 generic profiles from a different study.

4.7 Step 6 — the per-cell call

`compartment_opt` is mapped to `malignant` / `non_malignant`; if it is missing, `p_cnv > 0.5` is used instead. The result is joined back to the pipeline's cell ids through PREP's explicit cell map — so unlike the other callers, no id translation is needed downstream.

4.8 Step 7 — what gets published

File	Content
`*_numbat_calls.tsv`	one row per cell: `numbat_call`, `p_cnv`, `clone_opt`, `compartment_opt`, plus `sample_type`
`*_numbat_by_library.tsv`	malignant fraction per library, raw **and** lineage-gated
`*_numbat_by_sample_type.tsv`	the same pooled per compartment (tumour / blood / matched normal)
`segs_consensus_*.tsv`	one row per CNV **segment**: `cnv_state` (loh/amp/del/bamp), coordinates, LLR, support
`numbat/<individual>/`	posteriors, phylogeny, mutation graph; read by `NUMBAT_REPORT` for figures

The per-cell and per-segment counts answer different questions and differ by orders of magnitude — 9,227 cells against 66 segments on one real dataset.

4.9 Reading the two percentages

This is the most common misreading, so it is worth being blunt about.

Column	What it is	How to read it
`pct_malignant_raw`	Numbat's `compartment_opt`, nothing applied	**Cannot be ~0 for any compartment.** A high value here is not evidence of tumour
`pct_malignant_gated`	after the lineage gate; what the stratified compartments use	This is the result
`gate_overrode`	the difference	Reasonable to read as an empirical false-positive rate

A PBMC library at `11.4` raw and `0.3` gated is working correctly. A PBMC library high in **both** is a real problem.

5 Where the two callers meet

`modules/local/stratify/stratify_compartments.qmd` pools whatever evidence exists. Nothing is hardcoded to two callers: the tally counts whoever ran, which is what makes `--stratify_min_callers 2` meaningful once Numbat is on.

5.1 Each caller is normalised to a per-cell verdict

Numbat already is one. It arrives keyed by the pipeline's `cell_id`.

inferCNV arrives as a continuous score and is thresholded **per sample**:

```
cutoff = mean + k * SD   of that sample's own reference-lineage cells
         (k = --stratify_infercnv_sd, default 2)
call   = infercnv_score > cutoff
```

Per sample, not cohort-wide, because CNV-load distributions shift between samples — a CSF aspirate with a handful of tumour cells and a resection with thousands do not share a scale, and one global cutoff over-calls in some and under-calls in others. The cutoffs and the rule each one used are written to `data/infercnv_thresholds.csv`. A sample whose reference cells have **zero spread** is reported as a `quantile_degenerate_null` finding rather than silently accepted — `mean + 2 SD` would collapse to the mean and call everything above it malignant.

CopyKAT maps `aneuploid` → malignant, `diploid` → non-malignant. It is off by default (`skip_copykat = true`), so most runs have two callers, not three.

5.2 The consensus

Under `method = "consensus"` (the default) a **majority** verdict wins. When the callers split evenly, or when no caller scored the cell, **the annotation label decides** — a cell with no CNV evidence is not ambiguous, it is simply unmeasured. `evidence_basis` records which rule applied to each cell. Use `method = "cnv"` to exclude unmeasured cells instead.

With **zero** callers the split is the annotation relabelled, and the report says so in those words. With **one** caller, `--stratify_min_callers` has nothing to arbitrate and no disagreement can be reported.

5.3 The lineage gate

```
in_reference_lineage <- match_labels(label, reference_labels)
if (gate_on) evidence[in_reference_lineage == TRUE,
                      malignant_status := "non_malignant"]
```

On by default (`--stratify_gate_by_lineage`). A T cell is not malignant whatever the CNV score says. The override count is printed, written to `data/cnv_false_positive_by_lineage.csv` , and broken down by label — the single most useful number for judging whether the CNV calls can be trusted at all.

Numbat uses **the same predicate**, computed once in `numbat_prep.R` and carried on the cell map, so `pct_malignant_gated` means exactly what the final compartments mean. Two copies of one matching rule diverge silently; one copy cannot.

Note that `in_reference_lineage` is the **label test alone**. A reference-lineage cell whose cluster fell below the reference column floor is still a reference-lineage cell — it just did not help build the baseline. Conflating the two would understate the gate.

6 Side by side

	inferCNV	**Numbat**
Evidence	expression smoothed along the genome	allele imbalance at phased het SNPs, plus expression
Needs a BAM	no	**yes**, plus a ~10 GB panel
Default	**on**	**on** (needs a BAM; switched off with a warning when there is none)
Runs over	the whole object, once	one task per **individual**
Baseline	named cells (`ref_group_names`)	a profile matrix (`lambdas_ref`)
Reference cells scored?	no	**yes**
Sees copy-neutral LOH	**no**	yes
Chromosomes	autosomes only (1–22)	genome-wide
Output	continuous CNV load, thresholded in STRATIFY	a hard per-cell call
Can report “no tumour here”	yes, if no cell clears the cutoff	**no** — the heaviest clade is always `tumor`
Typical cost	hours, memory-heavy	pileup dominates; days on a pooled cohort

7 Parameters that change the answer

Parameter	Default	Effect
`--stratify_reference_labels`	per tumour type	**the baseline for both callers and the lineage gate.** The single highest-leverage setting here
`--celltype_col`	`sctype`	which column those labels are matched against
`--skip_infercnv`	`false`	—
`--cnv_analysis_mode`	`samples`	inferCNV HMM mode
`--cnv_infercnv_bayes_max_p_normal`	`0`	`0` disables the MCMC filter; see the note under *Step 5 — the actual calls*
`--cnv_infercnv_max_cells_per_group`	`0` (off)	capping; uncapped groups keep every cell
`--run_numbat`	`true`	needs the BAM and the phasing panel
`--numbat_group_col`	`patient_id`	**per-individual grouping.** Missing ⇒ per library ⇒ over-calls in normal libraries
`--numbat_cluster_col`	`seurat_clusters`	reference column granularity
`--numbat_ref_min_group_cells`	`100`	per-column floor; the author's own number
`--numbat_min_ref_cells`	`100`	below this, fall back to `ref_hca`
`--numbat_extra_reference_clusters`	`''`	per-cluster opt-in; never set by default
`--numbat_sample_type_col`	`sample_type`	enables the per-compartment table
`--numbat_min_llr`	`5`	CNV retention floor
`--numbat_max_entropy`	`0.5`	per-cell entropy ceiling
`--stratify_infercnv_sd`	`2`	inferCNV threshold, in SDs of the per-sample reference null
`--stratify_min_callers`	`1`	callers that must agree
`--stratify_gate_by_lineage`	`true`	the lineage gate

Neither `min_llr` nor `max_entropy` has been calibrated on the GBM cohort; both sit where Numbat puts them.

8 Traps worth knowing

A "CNV" split with no CNV caller. If every caller is skipped or every caller fails, STRATIFY still produces two compartments — from the annotation. The report says so explicitly. Check it before quoting a malignant fraction.

Reference labels that do not match the vocabulary. The matcher reconciles scType and Azimuth spellings, but a tumour type whose labels are simply absent (no `Fibroblast` in glioma) silently shrinks the baseline. The inferCNV notebook warns for every reference group named but unused; STRATIFY warns below 30 reference cells cohort-wide.

Numbat running per library. Visible as `Groups: N` equal to the library count in the PREP log, and as an explicit console warning. The malignant fractions it produces for normal libraries are not meaningful.

Reading `pct_malignant_raw` as the result. See *Reading the two percentages*. It is structurally non-zero.

A single-column expression reference. Numbat picks the best-fitting column per pseudobulk; one column means no choice at all. The run warns.

`S4-L1` versus `S4-L1Deeper`-style library pairs. If "Deeper" is a deeper resequencing of one GEM well, the same physical cell carries the same barcode in both, and namespacing makes it two cells. Nothing merges them automatically.

9 Worked example: GBM1-DFCI3

Seventeen GEX libraries, **one patient** (`DFCI3`), six timepoints, two compartments — 12 tumour (`TP`) and 5 peripheral blood (`NB`), supplied by `assets/metadata/gbm1_dfci3_exp_table.csv`.

What that configuration implies, end to end:

1. **One** Numbat group, not 17. `patient_id` must reach the merged object — `--input_exp_table` is what carries it.
2. **One** serial pileup across 17 libraries. This is the dominant cost.
3. An internal expression reference from this patient's own immune and stromal clusters, drawn from both compartments — PREP logs which: `[ref] DFCI3: baseline cells by sample type: NB=650, TP=410`.
4. The five blood libraries stay in the count matrix and are scored. Their `pct_malignant_raw` will not be near zero. Their `pct_malignant_gated` should be.
5. inferCNV runs once over all 17 libraries, but its threshold is rebuilt per sample from that sample's own reference cells.

The check to apply: open `cnv/data/<individual>_numbat_by_sample_type.tsv` and compare the **gated** column between `TP` and `NB`. A large gap is the expected shape. A small gap is the finding.